# Trabalho 1 - DevOps

Vitória de Lira Tenório - 800381

**Trabalho 1 - DevOps** **1**
**1. Visão Geral** **1**
**2. Tecnologias e Contêineres Utilizados** **2**
**3. Diagrama de containers do fluxo principal** **4**
**4. Endpoints da API** **4**
**5. Manual de Instalação - Versão com docker** **5**
5.1 Pré-requisitos 5
5.2 Obtenção do Código 5
5.3 Build e Inicialização 5
5.4 Verificação 6
5.5 Encerramento 6
**6. Manual de Instalação - Versão sem docker** **6**
6.1 Pré-requisitos 6
6.2 Instalação dos pré-requisitos 6
6.3 Configuração do banco de dados 7
6.4 Verificação do Redis 8
6.5 Obtenção do Código 8
6.6 Rodar backend 9
Iniciando a API 9
Iniciando o Worker 9
6.7 Rodar frontend 10

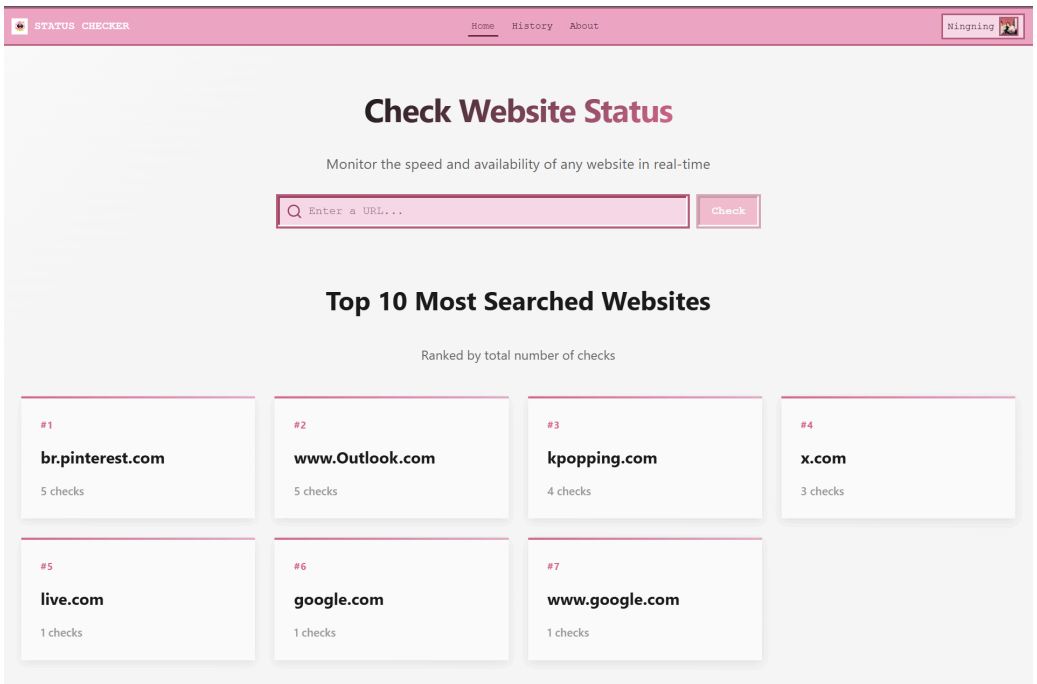
## 1. Visão Geral

O HeatLink é uma aplicação web para verificação de disponibilidade de URLs. O sistema recebe uma URL, verifica se há um resultado recente em cache e, caso não haja, enfileira a URL para processamento assíncrono por um worker dedicado. Os resultados são armazenados em banco de dados e podem ser consultados posteriormente, junto com um ranking das URLs mais verificadas.O fluxo principal da aplicação é o seguinte:

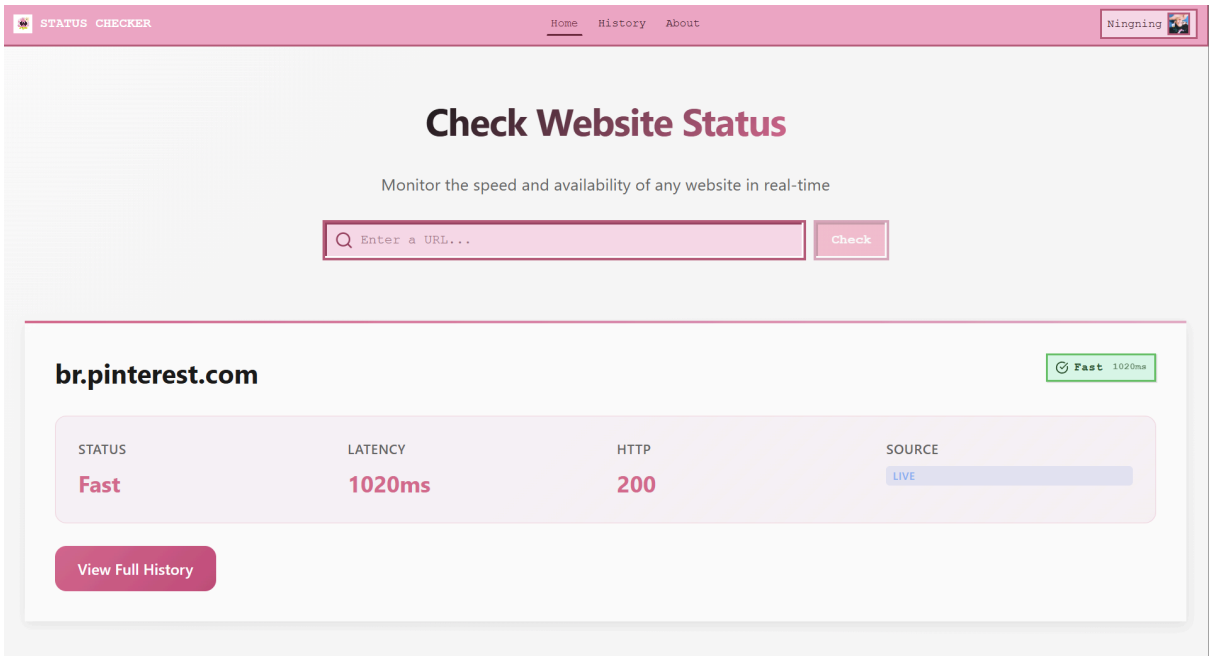
1. O usuário submete uma URL pelo frontend.
2. A API verifica se existe um resultado recente no cache Redis (cache hit). Em caso afirmativo, retorna imediatamente.
3. Se não houver cache (cache miss), a URL é enfileirada no Redis e a API retorna status "pendente".
4. O worker consome a fila, realiza a verificação HTTP da URL e persiste o resultado no PostgreSQL.

5. O usuário pode consultar o histórico de resultados e o ranking das URLs mais acessadas.

Home page




Resultado de pesquisa




O projeto está disponível no repositório público https://github.com/vitorialira92/devops/ e conta com duas branches, cada uma com um README dedicado.

## 2. Tecnologias e Contêineres Utilizados

A aplicação é composta por **cinco** contêineres orquestrados via Docker Compose:

- PostgreSQL 16 (Alpine)

Banco de dados relacional responsável pela persistência dos resultados de verificação. Armazena o histórico completo de cada URL verificada e os dados utilizados para geração do ranking. Os dados são persistidos em um volume Docker nomeado (postgres-data), garantindo que não sejam perdidos entre reinicializações.

- Redis 7 (Alpine)

  Atua em duas funções simultâneas: cache de resultados recentes e fila de mensagens. O cache evita requisições redundantes para URLs verificadas recentemente (TTL configurável, padrão de 30 segundos). A fila (url-queue) é implementada como uma lista Redis, onde a API insere URLs (LPUSH) e o worker as consome.

- API - Spring Boot (perfil api)

  Serviço backend responsável por expor os endpoints REST. Implementa a lógica de cache hit/miss, enfileiramento de URLs e consulta de resultados. Exposta na porta 8080. Desenvolvida em Java com Spring Boot, usando as bibliotecas Lombok, Jakarta Validation e Spring Data Redis.

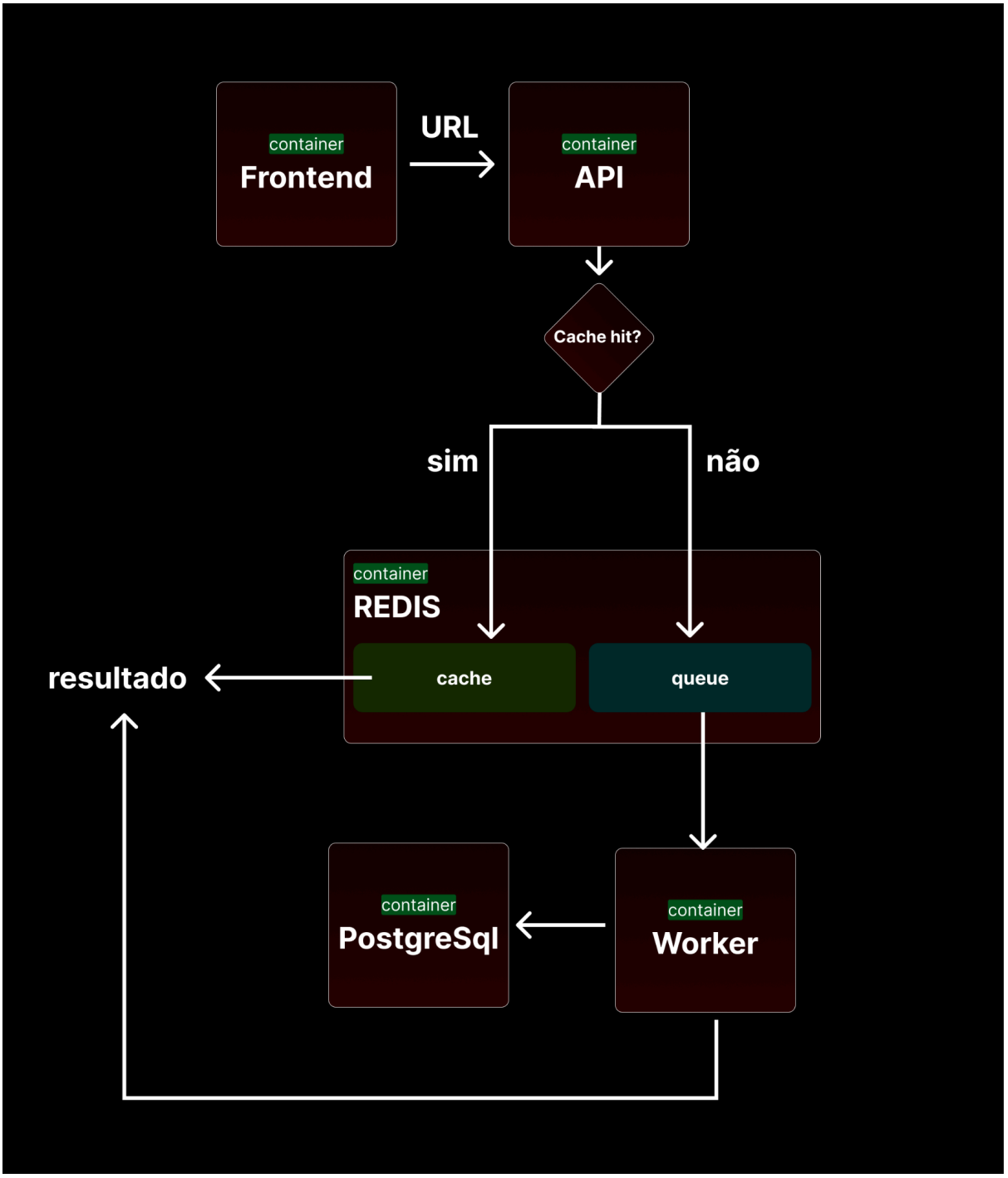
- Worker - Spring Boot (perfil worker)

  Instância separada do mesmo artefato backend, executada com o perfil Spring worker. Responsável por consumir a fila Redis, realizar as requisições HTTP às URLs enfileiradas, registrar o resultado (status HTTP, latência, disponibilidade) e armazenar no PostgreSQL. Não expõe portas externas.

- Frontend

  Interface web servida via Nginx, exposta na porta 3000. Permite ao usuário submeter URLs, acompanhar resultados e visualizar o ranking. Construída a partir do diretório heatlink-frontend.

## 3. Diagrama de containers do fluxo principal




## 4. Endpoints da API

Todos os endpoints estão sob o prefixo /api.

- **POST /api/check?url={url}**: Submete uma URL para verificação. Retorna o resultado imediatamente se houver cache válido (source: cache), ou enfileira para processamento e retorna status 202 Accepted (source: queued). URLs sem esquema recebem https:// automaticamente (apesar disso não acontecer quando submetida pelo frontend devido à normalização no frontend);

- **GET /api/results/url?url={url}:** Retorna o histórico completo de verificações de uma URL específica;
- **GET /api/ranking:** Retorna as 10 URLs mais consultadas no sistema.

# 5. Manual de Instalação - Versão com docker

## 5.1 Pré-requisitos

- Docker Engine 24 ou superior
- Docker Compose v2 ou superior
- Git

## 5.2 Obtenção do Código

Clone o repositório principal e os submódulos do backend e frontend:

```
None
git clone https://github.com/vitorialira92/devops.git
cd devops
git checkout t1-docker
```

Certifique-se de que a estrutura de diretórios está da seguinte forma:

```
heatlink/
├── compose.yml
├── heatlink.backend/
│   └── Dockerfile
└── heatlink-frontend/
    └── Dockerfile
```

## 5.3 Build e Inicialização

Execute o comando abaixo na raiz do projeto (onde está o compose.yml):

```
None
docker compose up --build
```

O Docker Compose irá:

- Construir as imagens do backend e do frontend.
- Iniciar o PostgreSQL e o Redis, aguardando seus healthchecks.
- Iniciar a API após banco e cache estarem saudáveis.
- Iniciar o Worker após a API ter subido.
- Iniciar o Frontend após a API estar disponível.

Para rodar em segundo plano (modo detached):

```
None
docker compose up --build -d
```

## 5.4 Verificação

Após a inicialização, os serviços estarão acessíveis em:
Frontend: http://localhost:3000
API REST: http://localhost:8080/swagger-ui/index.html#/ ou somente http://localhost:8080/

Para verificar o status dos contêineres:

```
None
docker compose ps
```

```
PS C:\Users\vitoria> docker ps
CONTAINER ID   IMAGE                COMMAND                  CREATED         STATUS                   PORTS                     NAMES
3873e819b599   t1-worker            "java -jar app.jar"      3 minutes ago   Up 2 minutes                                       heatlink-worker
efa51a6eed65   t1-frontend          "/docker-entrypoint.…"   3 minutes ago   Up 2 minutes             0.0.0.0:3000->80/tcp      heatlink-frontend
68d66218ebe7   t1-api               "java -jar app.jar"      3 minutes ago   Up 2 minutes             0.0.0.0:8080->8080/tcp    heatlink-api
20b5b7ef1ec2   redis:7-alpine       "docker-entrypoint.s…"   22 hours ago    Up 2 minutes (healthy)   6379/tcp                  heatlink-redis
5acb55170cdf   postgres:16-alpine   "docker-entrypoint.s…"   22 hours ago    Up 2 minutes (healthy)   5432/tcp                  heatlink-postgres
```

## 5.5 Encerramento

Para parar os contêineres mantendo os dados do banco:

```
None
docker compose down
```

Para parar e remover também o volume do banco de dados (todos os dados serão perdidos):

```
None
docker compose down -v
```

# 6. Manual de Instalação - Versão sem docker

## 6.1 Pré-requisitos

- Java 21
- Maven 3.9x ou superior
- Node.js 18 ou superior
- npm 9
- PostgreSQL 15
- Redis 7
- Git

## 6.2 Instalação dos pré-requisitos

Linux

```
None
sudo apt update
sudo apt install -y openjdk-21-jdk maven postgresql postgresql-contrib redis-server
sudo apt install redis-tools redis-server

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
```

macOs

```
None
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"

brew install openjdk@21 maven node@18 postgresql@15 redis

echo 'export PATH="/opt/homebrew/opt/openjdk@21/bin:$PATH"' >> ~/.zshrc
echo 'export PATH="/opt/homebrew/opt/node@18/bin:$PATH"' >> ~/.zshrc
source ~/.zshrc

brew services start postgresql@15
brew services start redis
```

Windows

- **Java 21**: Baixe o instalador em https://adoptium.net (JDK 21, Windows, x64, .msi). Durante a instalação, marque a opção de configurar as variáveis de ambiente automaticamente. Caso o JAVA_HOME não seja configurado corretamente após a instalação, defina-o manualmente apontando para a pasta raiz do JDK (ex: C:\Program Files\Java\jdk-21), nunca para o executável java.exe.
- **Maven**: Baixe o zip em https://maven.apache.org/download.cgi, extraia e adicione a pasta bin ao PATH.
- **Node.js**: Baixe o instalador LTS em https://nodejs.org. O npm é instalado junto.
- **PostgreSQL**: Baixe o instalador em https://www.postgresql.org/download/windows/ e siga o assistente.
- **Redis**: Baixe em https://github.com/tporadowski/redis/releases e execute o instalador.

## 6.3 Configuração do banco de dados

Linux e MacOs

```
None
sudo -u postgres psql        # Linux
psql postgres                # macOS

CREATE USER heatlink WITH PASSWORD 'heatlink';
CREATE DATABASE heatlink OWNER heatlink;
```

Windows
Abra o SQL Shell.

```
None
CREATE USER heatlink WITH PASSWORD 'heatlink';
CREATE DATABASE heatlink OWNER heatlink;
```

## 6.4 Verificação do Redis

Linux

```
None
sudo systemctl start redis-server
redis-cli ping   # deve retornar PONG
```

MacOs

```
None
brew services start redis
redis-cli ping   # deve retornar PONG
```

Windows

```
None
redis-cli ping # deve retornar PONG
```

```
PS C:\Users\vitoria> wsl
vitoria@LAPTOP-ADDILGVE:/mnt/c/Users/vitoria$ sudo apt update
sudo apt install redis-tools redis-server
[sudo] password for vitoria:
Hit:1 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:2 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:3 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [128 kB]
Hit:4 http://archive.ubuntu.com/ubuntu jammy-backports InRelease
Get:5 http://archive.ubuntu.com/ubuntu jammy-updates/main amd64 Packages [3439 kB]
Fetched 3567 kB in 3s (1167 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
62 packages can be upgraded. Run 'apt list --upgradable' to see them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
redis-server is already the newest version (5:6.0.16-1ubuntu1.1).
redis-tools is already the newest version (5:6.0.16-1ubuntu1.1).
redis-tools set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 62 not upgraded.
vitoria@LAPTOP-ADDILGVE:/mnt/c/Users/vitoria$ redis-cli ping
PONG
vitoria@LAPTOP-ADDILGVE:/mnt/c/Users/vitoria$
```

## 6.5 Obtenção do Código

Clone o repositório principal e os submódulos do backend e frontend:

```None
git clone https://github.com/vitorialira92/devops.git
cd devops
git checkout master
```

## 6.6 Rodar backend

```None
cd heatlink.backend
mvn clean package -DskipTests
```

Isso irá gerar o jar target/heatlink.backend-0.0.1-SNAPSHOT.jar.

### Iniciando a API

Abra um terminal que chamaremos de 1:

```None
# Linux / macOS
java -Dspring.profiles.active=api -jar target/heatlink.backend-0.0.1-SNAPSHOT.jar

# Windows (PowerShell)
java "-Dspring.profiles.active=api" -jar target\heatlink.backend-0.0.1-SNAPSHOT.jar
```

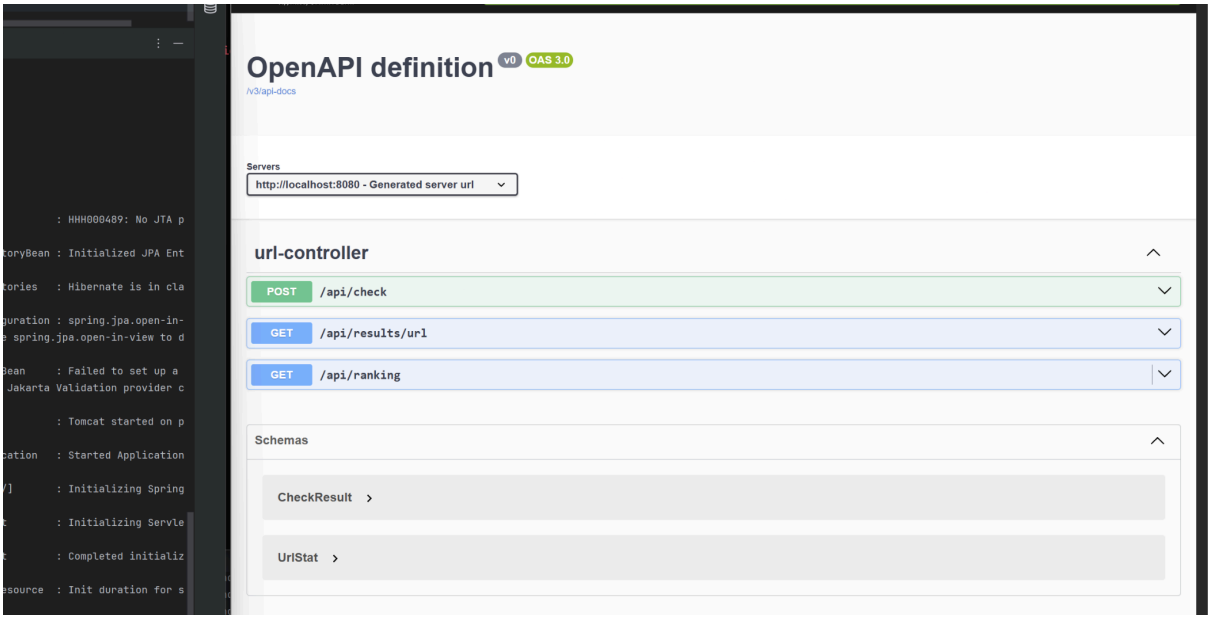
### Iniciando o Worker

Abra um outro terminal (2), mantendo o 1 aberto.

```None
# Linux / macOS
java -Dspring.profiles.active=worker -jar target/heatlink.backend-0.0.1-SNAPSHOT.jar

# Windows (PowerShell)
java "-Dspring.profiles.active=worker" -jar target\heatlink.backend-0.0.1-SNAPSHOT.jar
```

A api deve estar disponível em http://localhost:8080, também possível acessar através do swagger: http://localhost:8080/swagger-ui/index.html#/.




## 6.7 Rodar frontend

```
None
cd heatlink-frontend
npm install
npm run dev
```

O frontend é a última etapa e estará disponível em [http://localhost:5173](http://localhost:5173).